

Algorithms

Lecture #24

Syed Hamed Raza

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- The Huffman encoding algorithm is a **greedy algorithm**.
- The Huffman algorithm finds an **optimal** solution.

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- The Huffman codes provide a method of encoding data efficiently.
- Normally, when characters are coded using standard codes like ASCII.
- Each character is represented by a fixed-length codeword of bits, e.g., 8 bits per character.

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- consider the string "abacdaacac".
- if the string is coded with ASCII codes, the message length would be $10 \times 8 = 80$ bits.

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

Huffman Encoding Algorithm

- Determine the frequency of occurrence (relative probability) of each character in the message.
- This can be done by parsing the message.

character	a	b	c	d
frequency	5	1	3	1
probability	0.5	0.1	0.3	0.1

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- The greedy part:
 - Find two nodes with smallest frequency
 - Create a new node with these two nodes as children, and with weight equal to the sum of the weights of the two children
 - Continue until we have a single tree

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

1. Place the elements into minimum heap (by frequency).
2. Remove the first two elements from the heap.
3. Combine these two elements into one.
4. Insert the new element back into the heap.

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

```
3    pq.insert(t, freq[i])//priority queue
4
5  for i = 1 to N - 1
6  do x = pq.remove(); y=pq.remove()
7    z ← new TreeNode
8    z.left ← x; z.right ← y
9    z.freq ← x.freq + y.freq
10   pq.insert(z, z.freq);
11  return pq.remove(); // root
```

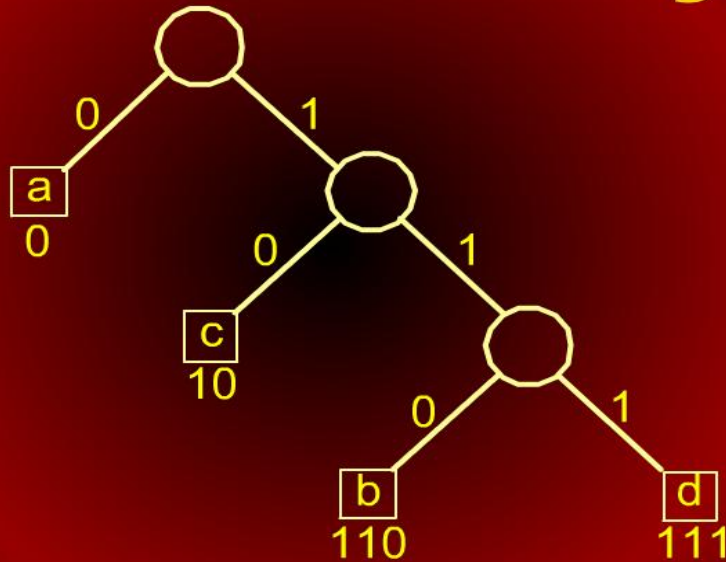

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

```
HUFFMAN(N, symbol[1..N], freq[1..N])  
1 for i = 1 to N  
2 do t ← TreeNode(symbol[i], freq[i])  
3   pq.insert(t, freq[i]) //priority queue  
4  
5 for i = 1 to N - 1  
6 do x = pq.remove(); y = pq.remove()  
7   z ← new TreeNode  
8   z.left ← x; z.right ← y
```

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding



Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

Prefix Code

The codewords assigned to characters by the Huffman algorithm have the property that no codeword is a prefix of any other:

character	a	b	c	d
frequency	5	1	3	1
probability	0.5	0.1	0.3	0.1
codeword	0	110	10	111

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- The prefix property is evident by the fact that codewords are leaves of the binary tree.
- Decoding a prefix code is simple.
- We traverse the root to the leaf letting the input 0 or 1 tell us which branch to take.



Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

Expected encoding length

- If a string of n characters over the alphabet $C = \{a, b, c, d\}$ is encoded using 8-bit ASCII, the length of encoded string is $8n$.
- For example, the string "abacdaacac" will require $8 \times 10 = 80$ bits.

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- The same string encoded with Huffman codes will yield

a	b	a	c	d	a	a	c	a	c
0	110	0	10	111	0	0	10	0	10

- This is just 17 bits, a significant saving!

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

Expected encoding length

- For a string of n characters over this alphabet, the expected encoded string length is

$$n(0.5 \cdot 1 + 0.1 \cdot 3 + 0.3 \cdot 2 + 0.1 \cdot 3) = 1.7n$$

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

Expected encoding length

- In general, let $p(x)$ be the probability of occurrence of a character,
- and let $d_T(x)$ denote the length of the codeword relative to some prefix tree T .

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- and let $d_T(x)$ denote the length of the codeword relative to some prefix tree T .
- The expected number of bits needed to encode a text with n characters is given by

$$B(T) = n \sum_{x \in C} p(x) d_T(x)$$

Greedy Algorithm: Huffman Encoding

Greedy Algorithm: Huffman Encoding

- Huffman algorithm uses a greedy approach to generate a prefix code T that minimizes the expected length $B(T)$ of the encoded string.
- In other words, Huffman algorithm generates an optimum prefix code.
- The question that remains is that why is the algorithm correct?

Huffman Encoding: Correctness

Huffman Encoding: Correctness

- Recall that the cost of any encoding tree T is

$$B(T) = n \sum_{x \in C} p(x) d_T(x)$$

- Our approach will be to show that any tree that differs from the one constructed by Huffman algorithm can be converted into one that is equal to Huffman's tree without increasing its costs.